- ***8*** The dimensions of the last layer are B × 3 × 51.

In this chapter we will do gradient descent manually, and to make this easier we have our Dist-DQN accept a single parameter vector called `theta` that we will unpack and reshape into multiple separate layer matrices of the appropriate sizes. This is easier since we can just do gradient descent on a single vector rather than on multiple separate entities. We also will use a separate target network as we did in chapter 3, so all we need to do is keep a copy of `theta` and pass that into the same `dist_dqn` function.

The other novelty here is the multiple output heads. We're used to a neural network returning a single output vector, but in this case we want it to return a matrix. To do that, we set up a loop where we multiply `l2` by each of three separate layer matrices, resulting in three different output vectors that we stack into a matrix. Other than that, it is a very simple neural network with a total of five dense layers.

Now we need a function that will take the output of our Dist-DQN, a reward, and an action, and generate the target distribution we want our neural network to get closer to. This function will use the `update_dist` function we used earlier, but it only wants to update the distribution associated with the action that was actually taken. Also, as you learned in chapter 3, we also need a different target when we've reached a terminal state. At the terminal state, the expected reward is the observed reward, since there are no future rewards by definition. That means the Bellman update reduces to $Z(s_t,a_t) \leftarrow R(S_t,A_t)$. Since we only observe a single reward, and there is no prior distribution to update, the target becomes what is called a *degenerate distribution*. That's just a fancy term for a distribution where all the probability mass is concentrated at a single value.

Listing 7.7. Computing the target distribution

```
def
get_target_dist(dist_batch,action_batch,reward_batch,support,lim=(-
\
        10,10),gamma=0.8):
    nsup = support.shape[0]
    vmin,vmax = lim[0],lim[1]
```